DR. RVR NRI DEEMED TO BE UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

Course	Python Programming	**Academic Year**	2026–2027
Student Name	R.Ramchran	**Branch & Section**	CSE (core)-1
Roll Number	26000054		

This report contains the ten Python programs supplied for the assignment. Each task includes the Python source code first, followed by its matching algorithm.

CASE STUDY INDEX

No.	Task Title	Main Concepts
1	Scholarship Eligibility Checker	Input, type conversion, conditional statements, logical operators
2	Medical Bill Discount Calculator	Input, arithmetic operations, if-else condition
3	Net Salary Calculator	Input, arithmetic operations, percentage calculation
4	Product Bill Discount Calculator	Input, multiplication, conditional statements
5	Event Cost Sharing Calculator	Input, arithmetic operations, conditional validation, rounding
6	Attendance Percentage and Extra Classes Calculator	Input validation, percentage calculation, if-else, while loop
7	ATM Menu-Based Banking System	while loop, menu-driven programming, if-elif-else, validation, break
8	Exam Performance and Grade Analyzer	Input, average calculation, chained comparisons, conditional grading
9	Classroom Seat Number Generator	Nested for loops, input validation, sequential numbering
10	Run-Length String Compression	Strings, loops, indexing, conditionals, string concatenation

Task 1: Scholarship Eligibility Checker

Concepts: Input, type conversion, conditional statements, logical operators

PYTHON SOURCE CODE

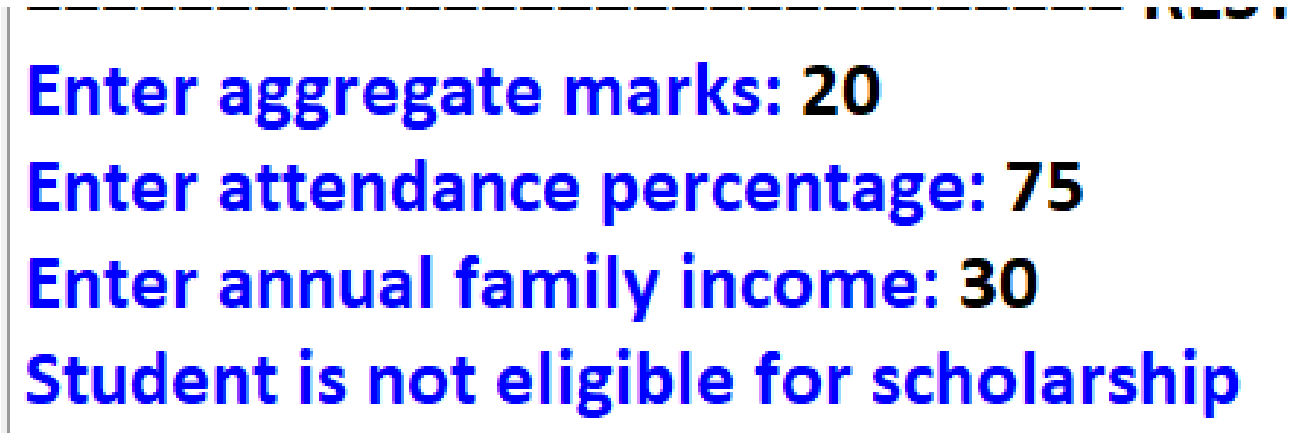
```
marks = float(input("Enter aggregate marks: "))
attendance = float(input("Enter attendance percentage: "))
income = float(input("Enter annual family income: "))

if marks >= 75 and attendance >= 75 and income <= 200000:
    print("Student is eligible for scholarship")
else:
    print("Student is not eligible for scholarship")
```

ALGORITHM

1. Start the program.
2. Read the student's aggregate marks, attendance percentage, and annual family income.
3. Check whether marks are at least 75, attendance is at least 75%, and annual family income is at most 200000.
4. If all three conditions are true, display that the student is eligible for a scholarship.
5. Otherwise, display that the student is not eligible for a scholarship.
6. Stop the program.

OUTPUT


Enter aggregate marks: 20
Enter attendance percentage: 75
Enter annual family income: 30
Student is not eligible for scholarship

Task 2: Medical Bill Discount Calculator

Concepts: Input, arithmetic operations, if-else condition

PYTHON SOURCE CODE

```
consultation = float(input("Enter consultation charge: "))
diagnostic = float(input("Enter diagnostic charge: "))
treatment = float(input("Enter treatment charge: "))

total = consultation + diagnostic + treatment

if total >= 5000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

print("Total Bill:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)
```

ALGORITHM

1. Start the program.
2. Read the consultation charge, diagnostic charge, and treatment charge.
3. Add the three charges to calculate the total bill.
4. If the total bill is at least 5000, calculate a 10% discount; otherwise, set the discount to zero.
5. Subtract the discount from the total bill to calculate the final bill.
6. Display the total bill, discount, and final bill.
7. Stop the program.

OUTPUT

```
Enter consultation charge: 20
Enter diagnostic charge: 30
Enter treatment charge: 50
Total Bill: 100.0
Discount: 0
Final Bill: 100.0
```

Task 3: Net Salary Calculator

Concepts: Input, arithmetic operations, percentage calculation

PYTHON SOURCE CODE

```
basic = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
overtime = float(input("Enter overtime amount: "))

gross = basic + allowance + overtime
deduction = gross * 0.10
net_salary = gross - deduction

print("Basic Salary:", basic)
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net_salary)
```

ALGORITHM

1. Start the program.
2. Read the basic salary, allowance, and overtime amount.
3. Add the basic salary, allowance, and overtime amount to calculate gross salary.
4. Calculate the deduction as 10% of the gross salary.
5. Subtract the deduction from the gross salary to calculate net salary.
6. Display the basic salary, gross salary, deduction, and net salary.
7. Stop the program.

OUTPUT

```
Enter basic salary: 30
Enter allowance: 50
Enter overtime amount: 70
Basic Salary: 30.0
Gross Salary: 150.0
Deduction: 15.0
Net Salary: 135.0
```

Task 4: Product Bill Discount Calculator

Concepts: Input, multiplication, conditional statements

PYTHON SOURCE CODE

```
price = float(input("Enter product price: "))
quantity = int(input("Enter quantity: "))

total = price * quantity

if total >= 2000:
    discount = total * 0.10
else:
    discount = 0

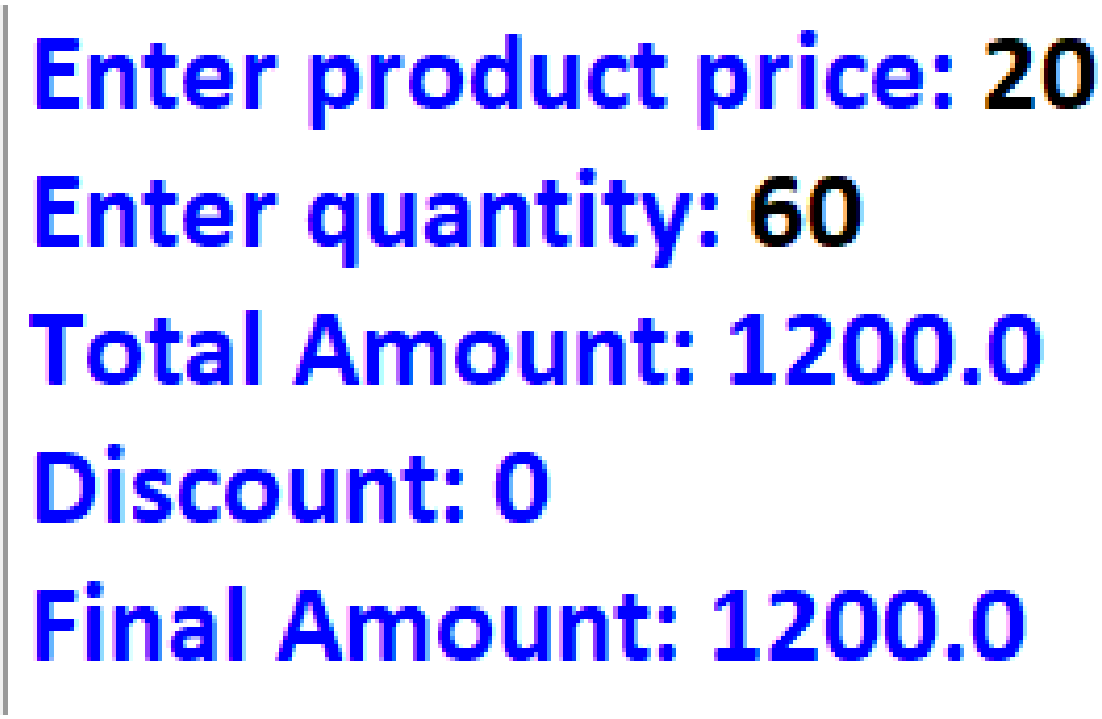
final_amount = total - discount

print("Total Amount:", total)
print("Discount:", discount)
print("Final Amount:", final_amount)
```

ALGORITHM

1. Start the program.
2. Read the product price and quantity purchased.
3. Multiply the price by the quantity to calculate the total amount.
4. If the total amount is at least 2000, calculate a 10% discount; otherwise, set the discount to zero.
5. Subtract the discount from the total amount to obtain the final amount.
6. Display the total amount, discount, and final amount.
7. Stop the program.

OUTPUT


Enter product price: 20
Enter quantity: 60
Total Amount: 1200.0
Discount: 0
Final Amount: 1200.0

Task 5: Event Cost Sharing Calculator

Concepts: Input, arithmetic operations, conditional validation, rounding

PYTHON SOURCE CODE

```
venue = float(input("Enter venue cost: "))
food = float(input("Enter food cost: "))
equipment = float(input("Enter equipment cost: "))
participants = int(input("Enter number of participants: "))

total_cost = venue + food + equipment

if participants > 0:
    contribution = total_cost / participants

    print("Total Event Cost:", total_cost)
    print("Contribution per Participant:", round(contribution, 2))
else:
    print("Number of participants must be greater than zero")
```

ALGORITHM

1. Start the program.
2. Read the venue cost, food cost, equipment cost, and number of participants.
3. Add the venue, food, and equipment costs to calculate the total event cost.
4. Check whether the number of participants is greater than zero.
5. If it is greater than zero, divide the total event cost by the number of participants and display the total cost and each participant's contribution rounded to two decimal places.
6. Otherwise, display a message that the number of participants must be greater than zero.
7. Stop the program.

OUTPUT

```
Enter venue cost: 20
Enter food cost: 30
Enter equipment cost: 80
Enter number of participants: 70
Total Event Cost: 130.0
Contribution per Participant: 1.86
```

Task 6: Attendance Percentage and Extra Classes Calculator

Concepts: Input validation, percentage calculation, if-else, while loop

PYTHON SOURCE CODE

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes conducted: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance details")
else:
    percentage = (attended / total) * 100

    print("Current Attendance:", round(percentage, 2), "%")

    if percentage >= 75:
        print("Attendance requirement satisfied")
    else:
        extra = 0

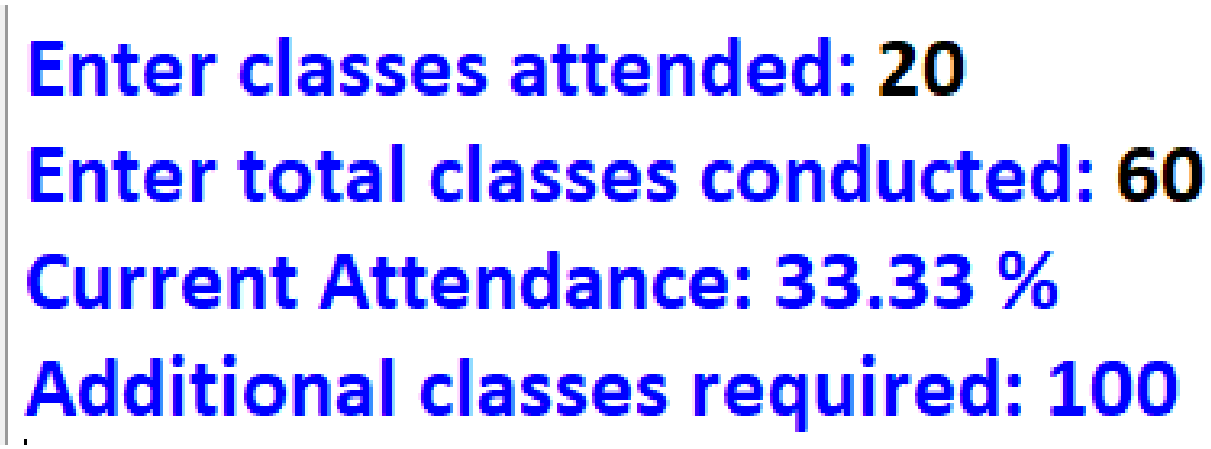
        while (attended + extra) / (total + extra) < 0.75:
            extra += 1

        print("Additional classes required:", extra)
```

ALGORITHM

1. Start the program.
2. Read the number of classes attended and the total number of classes conducted.
3. If total classes are not positive, or attended classes are negative or greater than total classes, display an invalid attendance message.
4. Otherwise, calculate and display the current attendance percentage.
5. If attendance is at least 75%, display that the attendance requirement is satisfied.
6. If attendance is below 75%, initialize the number of extra classes to zero.
7. Use a while loop to increase the extra-class count until (attended + extra) / (total + extra) is at least 0.75.
8. Display the additional classes required and stop.

OUTPUT


Enter classes attended: 20
Enter total classes conducted: 60
Current Attendance: 33.33 %
Additional classes required: 100

Task 7: ATM Menu-Based Banking System

Concepts: while loop, menu-driven programming, if-elif-else, validation, break

PYTHON SOURCE CODE

```
balance = 5000

while True:
    print("\\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        amount = float(input("Enter deposit amount: "))

        if amount > 0:
            balance += amount
            print("Deposit successful")
        else:
            print("Invalid amount")

    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))

        if amount <= 0:
            print("Invalid amount")
        elif amount <= balance:
            balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")

    elif choice == "3":
        print("Current Balance:", balance)

    elif choice == "4":
        print("Thank you for using the ATM")
        break

    else:
        print("Invalid choice")
```

ALGORITHM

1. Start the program and initialize the account balance to 5000.
2. Display the menu options: deposit, withdraw, check balance, and exit.
3. Read the user's choice.
4. For deposit, read the amount; add it to the balance only if it is greater than zero, otherwise display an invalid-amount message.
5. For withdrawal, read the amount; reject non-positive amounts, subtract it if it does not exceed the balance, or display an insufficient-balance message.
6. For check balance, display the current balance.
7. For exit, display a thank-you message and terminate the loop.
8. For any other choice, display an invalid-choice message.
9. Repeat the menu until the user chooses exit.

OUTPUT

```
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 2
Enter withdrawal amount: 30
Withdrawal successful
```

Task 8: Exam Performance and Grade Analyzer

Concepts: Input, average calculation, chained comparisons, conditional grading

PYTHON SOURCE CODE

```
mark1 = float(input("Enter first exam marks: "))
mark2 = float(input("Enter second exam marks: "))
mark3 = float(input("Enter third exam marks: "))

average = (mark1 + mark2 + mark3) / 3

print("Average Marks:", round(average, 2))

if mark1 < mark2 < mark3:
    print("Performance is improving")
elif mark1 > mark2 > mark3:
    print("Performance is declining")
else:
    print("Performance is fluctuating")

if average >= 75:
    print("Grade: A")
elif average >= 60:
    print("Grade: B")
elif average >= 40:
    print("Grade: C")
else:
    print("Grade: Fail")
```

ALGORITHM

1. Start the program.
2. Read marks from the first, second, and third exams.
3. Calculate the average by adding the three marks and dividing by three.
4. Display the average rounded to two decimal places.
5. Compare the three marks: if they strictly increase, display improving performance; if they strictly decrease, display declining performance; otherwise, display fluctuating performance.
6. Assign grade A for an average of at least 75, grade B for at least 60, grade C for at least 40, and Fail for an average below 40.
7. Display the grade and stop the program.

OUTPUT

```
Enter first exam marks: 30
Enter second exam marks: 80
Enter third exam marks: 60
Average Marks: 56.67
Performance is fluctuating
Grade: C
```

Task 9: Classroom Seat Number Generator

Concepts: Nested for loops, input validation, sequential numbering

PYTHON SOURCE CODE

```
rows = int(input("Enter number of rows: "))
columns = int(input("Enter number of columns: "))

if rows <= 0 or columns <= 0:
    print("Invalid classroom dimensions")
else:
    seat = 1

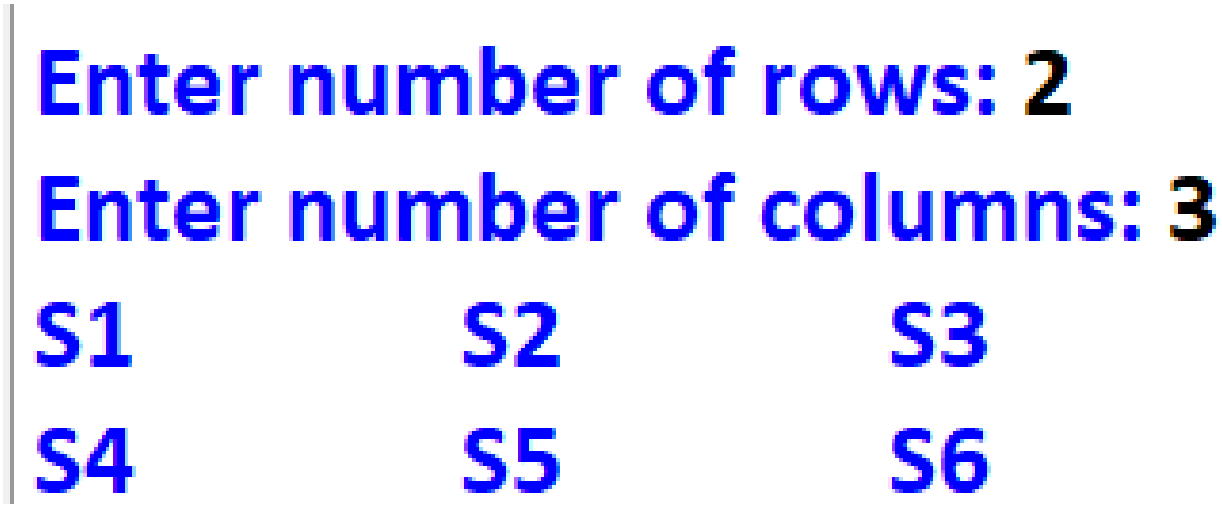
    for i in range(rows):
        for j in range(columns):
            print("S" + str(seat), end="\\t")
            seat += 1

        print()
```

ALGORITHM

1. Start the program.
2. Read the number of rows and columns in the classroom.
3. If either value is less than or equal to zero, display an invalid-dimensions message.
4. Otherwise, initialize the seat number to 1.
5. Use an outer loop to iterate through each row.
6. Inside the outer loop, use an inner loop to print the current seat label, beginning with S1, then increase the seat number by one.
7. After printing all seats in a row, move to the next line.
8. Continue until all rows and columns are printed, then stop.

OUTPUT


Enter number of rows: 2
Enter number of columns: 3
S1 S2 S3
S4 S5 S6

Task 10: Run-Length String Compression

Concepts: Strings, loops, indexing, conditionals, string concatenation

PYTHON SOURCE CODE

```
text = input("Enter a string: ")

if len(text) == 0:
    print("Empty string")
else:
    compressed = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            compressed += text[i - 1] + str(count)
            count = 1

    compressed += text[-1] + str(count)

    print("Original String:", text)
    print("Compressed String:", compressed)
    print("Original Length:", len(text))
    print("Compressed Length:", len(compressed))
```

ALGORITHM

1. Start the program and read a string from the user.
2. If the string is empty, display an empty-string message.
3. Otherwise, initialize the compressed string as empty and the character count to one.
4. Loop through the string from the second character to the end.
5. If the current character matches the previous character, increase the count.
6. If it differs, append the previous character and its count to the compressed string, then reset the count to one.
7. After the loop, append the final character and its count.
8. Display the original string, compressed string, original length, and compressed length.
9. Stop the program.

OUTPUT

```
Enter a string: 3
Original String: 3
Compressed String: 31
Original Length: 1
Compressed Length: 2
```